

用户证据应该留在上下文，还是写入模型参数？

个性化模型的方法设计笔记

2026 年 8 月 29 日

1 结论

新出现的用户偏好，先放在提示词或外部记忆里。这样容易查看、修改和删除。

只有满足下面几项时，才值得训练用户适配器（adapter）：

- 同一偏好在多个会话中反复出现；
- 偏好长期稳定；
- 偏好会影响多类任务；
- 即使不提供用户档案，偏好也应该生效。

参数更新不是默认答案。它必须比上下文更有效，同时不能降低任务成功率。

本文提出一个简单实验。对同一批用户证据，做两个版本：

1. **上下文版**：把证据保存在提示词或外部记忆中，不改模型参数；
2. **参数版**：用同一批证据训练用户适配器，测试时不再提供这批证据。

两个版本使用同一个起点，并在同一组后续任务上比较。只有参数版的个性化效果更好，且任务成功率不低于上下文版，才选择参数版。否则保留上下文。

这仍是一个待验证的方案，不是已经成立的新算法。

2 要区分的三种信息

表 1: 三种信息的作用不同。

信息	用法	特点
提示词或外部记忆	在运行时提供用户事实、偏好或历史记录。	容易修改，但会占用上下文，也可能检索错误。
训练提示	训练时给教师模型的提示，用来生成训练目标。	它是训练信号，不一定是部署时的用户状态。
用户适配器 $\Delta\theta_u$	针对用户 u 学到的一组参数。	不需要重复输入用户档案，但更难检查和修改。

这里的在线更新指部署后更新。系统先服务用户，再用新交互更新参数，并得到新的适配器。用固定数据集训练、只是在训练时重新采样，并不等于部署后学习。

本文讨论的个性化智能体还必须完成任务。只改变文字风格，不等于个性化智能体。

3 这些设计从哪里来

当前代码可能有错误，也可能继续变化，因此不作为方法依据。本文只使用论文中已经说明的设计和结果。

表 2: 设计依据。

设计	依据	本文如何使用
按时间划分数据	SPRInG 把每个用户的记录按时间分成五个阶段。每个阶段前 90% 用于更新，后 10% 用于评估 [3]。	直接采用
比较上下文和参数	OPPU 比较检索、用户档案提示和每用户 PEFT [1]。SPRInG 也分别评估适配器和检索。	作为必要对照
比较外部经验和参数学习	EvolveR 的主要方法把经验当作上下文。它还测试了把经验写入参数的版本 [4]。	作为研究动机
任务成功率	EvolveR 用最终答案是否正确作为任务结果奖励。我们的智能体也必须先完成任务。	作为独立指标
安全检查	SPRInG 只把安全过滤和投毒防护列为未来工作。	不放进核心方法
置信区间	SPRInG 用它报告实验结果，但不用它决定每次更新。	只用于结果报告
回退	上述论文没有给出可直接继承的回退方法。	暂不加入

4 现有方法已经做了什么

OPPU。 OPPU 用每个用户的历史数据训练一个 PEFT 模块，也可以同时使用检索或用户档案 [1]。它是离线方法，不处理部署后的逐次更新。

Personalized DPO。 Personalized RLHF 从离线偏好数据学习用户表示和 LoRA 参数 [2]。它没有研究外部记忆，也没有研究智能体的任务结果。

SPRInG。 SPRInG 是最接近的工作 [3]。它同时使用用户 LoRA 和检索缓冲区，并按时间持续更新。因此，“外部记忆加适配器”不是新贡献。

SPRInG 根据似然变化选择训练样本。更新后仍然难学的样本进入缓冲区。它没有对同一批证据分别训练上下文版和参数版，再用后续任务选择其中一个。

EvolveR。 EvolveR 让智能体从轨迹中总结经验，并把经验放在外部经验库 (Experience Base) 中 [4]。它还测试了直接学习这些经验。论文报告的平均分从 0.382 降到 0.371。这说明，未经筛选就把经验写入参数，可能带来噪声。

CoSteer。 CoSteer 不训练用户参数 [5]。本地小模型分别读取和不读取用户上下文，两次输出的差值用来调整云端模型。它说明运行时方法也可能有效，因此应作为对照。

5 我们要验证的方法

设用户 u 当前有适配器 $\Delta\theta_u^{(t)}$ 和外部记忆 $M_u^{(t)}$ 。新证据 E_t 来自该用户之后的交互。

上下文版。

$$M_{u,\text{ctx}} = \text{MemoryUpdate}(M_u^{(t)}, E_t), \quad \Delta\theta_{u,\text{ctx}} = \Delta\theta_u^{(t)}. \quad (1)$$

参数版。

$$\Delta\theta_{u,\text{par}} = \text{TrainAdapter}(\Delta\theta_u^{(t)}, E_t), \quad M_{u,\text{par}} = M_u^{(t)}. \quad (2)$$

参数版和上下文版必须使用同一批证据、同一个起点和同一组后续任务 V_t 。参数版测试时不能再把 E_t 放进提示词，否则无法知道参数学到了什么。

选择规则只有两项：

$$\begin{aligned} U_{\text{pref}}(m_{\text{par}}; V_t) &> U_{\text{pref}}(m_{\text{ctx}}; V_t), \\ U_{\text{task}}(m_{\text{par}}; V_t) &\geq U_{\text{task}}(m_{\text{ctx}}; V_t). \end{aligned} \quad (3)$$

两项都满足，选择参数版。否则选择上下文版。若证据互相矛盾或数量太少，暂不更新。

这条规则是本文提出的实验方案，不是从现有论文直接复制的结论。

6 实验怎么做

6.1 按时间划分

先按完整会话排序，再提取用户档案或生成训练数据：

$$H_u^{\text{past}} \prec E_t \prec V_t \prec H_u^{\text{test}}. \quad (4)$$

过去数据用于初始化， E_t 用于生成两个版本， V_t 用于选择，最后一段只用于测试。

6.2 最少需要的对照

至少分别报告三类结果：

- 后续任务中的个性化效果；
- 环境可以验证的任务成功率；
- 提示词、训练和推理成本。

不要把它们合成一个总分。置信区间只用于说明结果是否稳定，不参与每次更新。

表 3: 实验对照。

版本	回答的问题
基础模型	不做个性化时表现如何?
基础模型 + 提示词/记忆	只用上下文能做到什么?
仅适配器	参数是否真的学到用户偏好?
适配器 + 提示词/记忆	两种方式能否互补?
OPPU	普通每用户 PEFT 是否足够?
SPRInG	已有的持续更新加检索是否足够?
CoSteer (条件允许时)	不训练参数能否达到相近效果?
本文方案	按证据选择保存位置是否更好?

7 创新边界

以下内容已有工作做过，不能作为新贡献：

- 每个用户一个适配器；
- 持续更新用户适配器；
- 同时使用外部记忆和适配器；
- 比较外部上下文和参数学习。

可能的新点更窄：对同一批证据，生成上下文版和参数版，再用同一用户的后续任务决定证据应该放在哪里。

这个差异仍需更完整的文献检查和实验验证。当前只能称为研究假设。

8 下一步

1. 选择带时间信息的同一用户数据。
2. 根据论文或官方代码实现 OPPU、SPRInG 和提示词/记忆对照。
3. 固定一种适配器训练方法和一种外部记忆写入方法。
4. 先跑基础模型、仅上下文、仅适配器和两者结合。
5. 再跑本文的配对比较。
6. 个性化和任务成功都通过后，再扩展到真实工具调用的智能体任务。

参考文献

- [1] Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-Tuning. EMNLP, 2024. <https://aclanthology.org/2024.emnlp-main.372/>.
- [2] Xinyu Li, Ruiyang Zhou, Zachary C. Lipton, and Leqi Liu. Personalized Language Modeling from Personalized Human Feedback. arXiv:2402.05133, 2024. <https://arxiv.org/abs/2402.05133>.
- [3] Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM Personalization via Selective Parametric Adaptation and Retrieval-Interpolated Generation. arXiv:2601.09974, 2026. <https://arxiv.org/abs/2601.09974>.
- [4] Rong Wu, Xiaoman Wang, Jianbiao Mei, and others. EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle. arXiv:2510.16079, 2025. <https://arxiv.org/abs/2510.16079>.
- [5] Hang Lv, Sheng Liang, Hao Wang, and others. CoSteer: Collaborative Decoding-Time Personalization via Local Delta Steering. arXiv:2507.04756, 2025. <https://arxiv.org/abs/2507.04756>.